

Function Calling

Understanding Function Calling



在現代 AI 中，Function Calling 賦予大型語言模型（LLMs）與外部工具協作的的能力，具體體現在以下模型能力：

- **理解工具清單（Function schema）**：模型能透過工具定義清單，清楚知道有哪些可用工具及其功能與輸入輸出的格式。
- **判斷輸入意圖**：根據用戶輸入，決定是否需要調用外部工具，例如：
 - 獲取資訊（如查詢資料庫、調用第三方 API）。
 - 執行操作（如發送通知、寫入資料）。
- **生成結構化輸出（Structured JSON）**：模型會根據判斷後，產生一個符合規定的 JSON 格式請求，並提供給外部系統進行處理。

當我們提到某個 AI 服務具備 Function Calling 能力時，除了上述模型本身具備的能力外，通常也包含了一套外部系統（工具執行者）的配合，其功能包括：

- **檢查輸出格式（Output validation）**：
 - 解析並驗證 LLM 產出的 JSON 是否符合工具調用的要求格式。
- **執行工具函數（Tool execution）**：
 - 根據 JSON 請求執行相應函數，例如呼叫 API、查詢或操作資料庫。
- **整合執行結果為上下文（Context integration）**：
 - 將函數的執行結果回傳給模型，供模型作為新上下文，生成最終完整的回應給用戶。

需要注意的是，模型本身並不直接執行這些外部工具，而是透過結構化輸出（JSON）和外部工具執行系統協作。這種彈性使得 Function Calling 成為建構 AI 代理、自動化工作流程以及企業級系統整合的重要基礎。

然而，目前不同廠商或模型提供的 Function Calling 實作方式尚未標準化，增加了整合開發成本。因此像 MCP（Model-Controller-Plugin）等協議開始出現，旨在提供一致的規範標準，使不同的模型與外部工具更容易相互協作，降低系統間的整合難度並提升工具互通性。

範例：

例如，當用戶詢問：「台北今天的天氣如何？」

模型可能產生以下結構化輸出來調用外部 API：

```
json
複製編輯
{
  "function_call": {
    "name": "get_weather",
    "arguments": {
      "location": "台北",
      "date": "2025-04-22"
    }
  }
}
```

外部系統取得此輸出後，呼叫對應 API 查詢天氣，再將結果回傳給模型，使模型回覆給用戶：「台北今天是多雲，氣溫介於22°C到27°C之間。」

簡介

Function Calling 是大型語言模型（LLMs）與外部工具交互的關鍵技術，允許模型根據輸入意圖判斷是否需要調用工具，並生成結構化的調用請求（通常為 JSON 格式）。這些請求由外部系統執行，結果融入上下文，幫助模型完成任務。Function Calling 並非讓 LLM “自己”調用函數，而是讓模型專注於理解意圖和生成調用格式，外部系統負責實際執行。它是 AI 代理（Agent）的核心組件，並與標準化協議（如 MCP）協同工作，廣泛應用於企業自動化、數據查詢和操作執行。

源起

Function Calling 起源於 LLMs 的局限性：早期模型僅能生成文字，無法直接存取外部數據或執行操作。2020 年代初，隨著 GPT-3.5 和 GPT-4 的進展，OpenAI 引入 Function Calling（2023 年），標準化了工具調用流程，讓 LLM 能生成結構化請求。此前，開發者依賴自訂插件或 API 整合，但這些方案缺乏統一性，導致高整合成本。Anthropic（Claude）、Google（Gemini）等公司隨後推出類似功能，推動 Function Calling 成為 AI 服務的標配。值得注意的是，所有 LLM 服務（如 ChatGPT.com）都包裹了一個外部系統，負責處理輸入、調用工具並管理上下文，Function Calling 因此成為模型與外部系統協作的橋樑。

說明

Function Calling 的本質在於 LLM 和外部系統的分工合作：

- **LLM 的角色：**LLM 負責理解用戶輸入的意圖，判斷是否需要調用工具來獲取資訊或執行操作，並生成符合調用格式的輸出（通常為 JSON 結構）。例如，當用戶輸入“查詢員工 A 的業績”，LLM 根據工具清單（schema）生成：

```
{
  "function": "get_sales_data",
  "arguments": {
    "user_id": "A",
    "month": "2025-04"
  }
}
```

LLM 並不直接執行函數，而是將調用請求交給外部系統。

- **外部系統（工具執行者）的角色：**外部系統檢查 LLM 的輸出是否包含工具調用請求，若有，則執行相應函數並將結果融入上下文，供 LLM 繼續生成回應。這個系統的設計因服務架構而異，無統一規範，但其核心功能包括：
 - 解析 LLM 的 JSON 輸出，驗證調用格式。
 - 執行工具（例如查詢資料庫、調用 API）。
 - 將結果作為上下文返回給 LLM。
- **運作流程：**
 1. 開發者定義工具清單，描述工具名稱、功能和參數（JSON Schema）。
 2. 用戶輸入和工具清單作為上下文傳遞給 LLM。
 3. LLM 判斷是否需要工具，生成結構化調用請求。
 4. 外部系統執行調用，返回結果，LLM 根據結果生成最終回應。

Function Calling 的優勢在於其靈活性：它支援任何工具（從資料庫查詢到發送通知），無功能限制，且能透過自訂 API 實現動態探索（dynamic discovery）。然而，其挑戰在於非標準化（不同 LLM 使用不同格式）和開發成本（需自訂執行邏輯）。這些問題促使 MCP 等協議的出現，標準化工具交互。

實例

以企業業績系統為例，假設一家公司需要查詢員工銷售數據。開發者定義一個 `get_sales_data` 工具，連接到 PostgreSQL 資料庫。當用戶輸入“查詢員工 A 在 2025 年 4 月的業績”，LLM 理解意圖，根據工具清單生成以下調用：

```
{
  "function": "get_sales_data",
  "arguments": {
    "user_id": "A",
    "month": "2025-04"
  }
}
```

外部系統解析此 JSON，執行資料庫查詢，返回結果（例如銷售額清單）。結果作為上下文傳回 LLM，生成最終回應：“員工 A 在 2025 年 4 月的總銷售額為 \$10,000。”此場景展示 Function Calling 如何讓 LLM 專注於意圖理解，外部系統負責執行，實現自動化查詢，適用於多用戶、多系統的企業環境。

動手作

以下是一個使用 Node.js 和 OpenAI 的 Function Calling 示例，模擬業績系統的資料庫查詢。請確保安裝 Node.js (≥18)，並設置 OpenAI API 金鑰和 PostgreSQL 資料庫。

1. 安裝依賴：

```
npm install openai pg
```

2. 實現程式碼 (`function_calling.js`)：

```
const { OpenAI } = require("openai");
const { Pool } = require("pg");

// 初始化 OpenAI 和 PostgreSQL
const openai = new OpenAI({ apiKey: "your-api-key" });
const pool = new Pool({
  user: "postgres",
  password: "secret",
  host: "localhost",
  port: 5432,
  database: "sales"
});

// 定義工具清單
```

```

const tools = [
  {
    type: "function",
    function: {
      name: "get_sales_data",
      description: "Query sales data for a user and month",
      parameters: {
        type: "object",
        properties: {
          user_id: { type: "string", description: "Employee ID" },
          month: { type: "string", description: "Month in YYYY-MM format" }
        }
      },
      required: ["user_id", "month"]
    }
  }
];

```

// 執行工具調用

```

async function executeToolCall(call) {
  if (call.function.name === "get_sales_data") {
    const { user_id, month } = JSON.parse(call.function.arguments);
    const client = await pool.connect();
    try {
      const result = await client.query(
        "SELECT * FROM sales WHERE user_id = $1 AND month = $2",
        [user_id, month]
      );
      return JSON.stringify({ results: result.rows });
    } finally {
      client.release();
    }
  }
  return "Unknown function";
}

```

// 主函數

```

async function main() {
  const response = await openai.chat.completions.create({
    model: "gpt-4",
    messages: [{ role: "user", content: "查詢員工 A 在 2025 年 4 月的業績" }],
    tools: tools,
    tool_choice: "auto"
  });

  const choice = response.choices[0];
  if (choice.finish_reason === "tool_calls") {
    const toolCall = choice.message.tool_calls[0];
    const result = await executeToolCall(toolCall);
    console.log("Tool result:", result);
    // 可將結果傳回 LLM 繼續生成
    const finalResponse = await openai.chat.completions.create({
      model: "gpt-4",
      messages: [
        { role: "user", content: "查詢員工 A 在 2025 年 4 月的業績" },
        { role: "assistant", content: null, tool_calls: [toolCall] },
        { role: "tool", content: result, tool_call_id: toolCall.id }
      ]
    });
    console.log("Final response:", finalResponse.choices[0].message.content);
  } else {
    console.log("Response:", choice.message.content);
  }
}

main().catch(console.error);

```

3. 運程式碼：

```
node function_calling.js
```

說明：

- 程式碼實現 `get_sales_data` 工具，連接到 PostgreSQL，模擬業績查詢。

- LLM 生成工具調用，外部系統（工具執行者）執行查詢並返回結果。
- 結果傳回 LLM，生成最終回應，展示你的表達（LLM 專注於調用格式，外部系統負責執行）。
- 你可擴展工具清單（例如新增 `send_notification`），或實現動態探索（透過 `/list_tools` API）。

總結

在現代 AI 中，Function Calling 賦予大型語言模型（LLMs）與外部工具協作的 ability，具體體現在以下模型能力：

- **理解工具清單**：模型知道可用的工具及其功能（透過工具清單定義）。
- **判斷輸入意圖**：根據用戶輸入，決定是否需要調用工具，可能是為了獲取資訊（例如查詢資料庫）或執行操作（例如發送通知、儲存資料）。
- **生成結構化輸出**：產生符合調用格式的 JSON 請求，供外部系統處理。

當我們說一個 AI 服務具備 Function Calling 能力時，除了上述模型能力，還包括一個外部系統（工具執行者），其功能包括：

- **檢查輸出格式**：解析 LLM 的 JSON 輸出，驗證是否為有效工具調用。
- **執行工具**：根據調用請求執行相應函數（例如查詢 API、操作資料庫）。
- **整合上下文**：將執行結果作為上下文傳回 LLM，讓模型生成最終回應。

Function Calling 的靈活性使其成為 AI 代理和企業自動化的核心，但其非標準化（不同模型格式不同）和開發成本促使 MCP 等協議的發展，標準化工具交互，提升互通性。

參考

https://grok.com/share/bGVnYWN5_a51be2b2-6676-4f70-a453-6b093af417bb